#100DaysOfMySQL | DAY 19

MySQL Learning Journey

Database Normalization & Entity-Relationship (ER) Modeling

Data Redundancy | Anomalies | 1NF, 2NF, 3NF | BCNF, 4NF, 5NF | ER Model Basics | Entities & Attributes | Relationships & Cardinality

A simple, example-driven recap of everything covered today — written in plain language with tables, symbols and worked examples.

Part 1 — Database Normalization

1. What is Data Redundancy?

Data redundancy means the same piece of data is stored in more than one place in a database. It happens when tables aren't designed carefully, and it wastes storage while making the data harder to keep consistent.

EXAMPLE

Notice how the customer's `City` and `Phone` repeat for every order they place — that repetition is redundancy.

OrderID	CustomerName	City	Phone	Product
101	Ravi Kumar	Vijayawada	9876543210	Keyboard
102	Ravi Kumar	Vijayawada	9876543210	Mouse
103	Ravi Kumar	Vijayawada	9876543210	Monitor

Ravi's city and phone are stored three times for no reason. If Ravi moves to a new city, we now have to update three rows instead of one.

2. Types of Anomalies

Redundant, badly-structured tables lead to three classic problems, called **anomalies**:

Anomaly	Simple Meaning	Example (using table above)
Insertion Anomaly	We are unable to add certain data unless some unrelated data is also available.	We can't add a new customer's details unless they place an order first, because there's no order row to attach them to.
Update Anomaly	The same data exists in multiple rows, so updating it means changing it everywhere — miss one, and the data becomes inconsistent.	If Ravi changes his phone number, we must update it in all 3 rows. Missing even one row leaves incorrect data.
Deletion Anomaly	Deleting one row accidentally deletes other useful data too.	If we delete order 101 (Ravi's only remaining order) we may lose his City/Phone details entirely.

3. What is Normalization?

Normalization is the step-by-step process of organizing columns and tables in a database to reduce data redundancy and remove anomalies — by splitting large, messy tables into smaller, related tables.

Main Goals of Normalization

✅ **Remove data redundancy** — store each fact only once.

✅ **Ensure proper data dependencies** — every column should depend only on the table's key (logical grouping of data).

✅ **Improve data integrity** — keep data accurate and consistent across the database.

4. Normal Forms

Normalization is done in stages, called **Normal Forms (NF)**. Each form fixes a specific kind of problem, and every higher form must also satisfy the rules of the lower ones.

1NF → **2NF** → **3NF** → **BCNF** → **4NF** → **5NF**

1NF First Normal Form

Rule: Every column must hold only atomic (single, indivisible) values — no multiple values in one cell, and no repeating groups of columns.

BEFORE — violates 1NF

StudentID	Name	Subjects
1	Anitha	Maths, Science, English

AFTER — satisfies 1NF

StudentID	Name	Subject

1	Anitha	Maths
1	Anitha	Science
1	Anitha	English

2NF Second Normal Form

Rule: Must already be in 1NF, AND every non-key column must depend on the *entire* primary key (this matters only when the key is made of more than one column — no **partial dependency** allowed.)

BEFORE — violates 2NF

Primary key = (StudentID, CourseID). But `StudentName` depends only on `StudentID`, not on the whole key — that's a partial dependency.

StudentID	CourseID	StudentName	CourseFee
1	C101	Anitha	5000
1	C102	Anitha	7000

AFTER — satisfies 2NF (split into two tables)

StudentID	StudentName
1	Anitha

CourseID	CourseFee
C101	5000
C102	7000

3NF Third Normal Form

Rule: Must already be in 2NF, AND no non-key column should depend on another non-key column (no **transitive dependency**). Every column should depend directly on the key, and nothing but the key.

BEFORE — violates 3NF

`City` depends on `ZipCode`, and `ZipCode` depends on `StudentID` — so `City` depends on the key only indirectly (transitively).

StudentID	Name	ZipCode	City
1	Anitha	520001	Vijayawada

AFTER — satisfies 3NF (split ZipCode/City out)

StudentID	Name	ZipCode
1	Anitha	520001

ZipCode	City
520001	Vijayawada

BCNF Boyce-Codd Normal Form

Rule: A stricter version of 3NF. For every functional dependency `A → B`, `A` must be a **super key**. It fixes rare cases where a table is in 3NF but still has anomalies because of overlapping candidate keys.

4NF Fourth Normal Form

Rule: Must be in BCNF, AND have no **multi-valued dependency** — i.e., a table should not contain two or more independent multi-valued facts about the same entity (e.g., a student's list of hobbies and list of languages should not sit in one combined table).

5NF Fifth Normal Form

Rule: Also called **Project-Join Normal Form (PJNF)**. A table is broken down into smaller tables so it can always be reconstructed by joining them back together, without losing or creating extra data (no **join dependency** issues).

> **Today's focus:** I studied 1NF, 2NF and 3NF in depth with worked table examples, and covered BCNF, 4NF, 5NF at a conceptual level.

Part 2 — Entity-Relationship (ER) Model

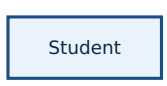
1. What is an ER Model?

The **Entity-Relationship (ER) Model** is a visual way of designing a database before creating actual tables. It shows what real-world "things" (entities) exist, what details (attributes) they have, and how they connect to each other (relationships) — usually drawn as an **ER Diagram**.

2. What is an Entity?

An **entity** is any real-world object or concept that has data stored about it — for example, a *Student*, a *Course*, or an *Employee*. Each individual occurrence (e.g., one specific student) is called an **entity instance**.

Types of Entities



Strong Entity
Has its own primary key; can exist independently. Drawn as a single rectangle.

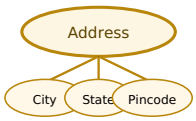
Weak Entity
Has no primary key of its own; depends on a strong entity. Drawn as a double rectangle.

3. What are Attributes?

An **attribute** is a property or detail that describes an entity — e.g., a Student entity can have attributes like `StudentID`, `Name`, and `DateOfBirth`.

Types of Attributes & Their Symbols

Simple / Atomic
Cannot be divided further. Drawn as a plain oval.



Composite
Can be split into smaller sub-attributes. Connected ovals branching from a main oval.

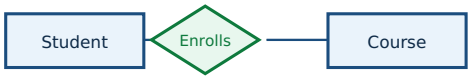
Derived
Calculated from another attribute. Drawn as a dashed oval.

Multi-valued
Can hold more than one value at once. Drawn as a double oval.

Key Attribute
Uniquely identifies each entity instance. Name is underlined.

4. Relationships & Their Types

A **relationship** shows how two or more entities are connected/associated with each other. It is drawn as a **diamond** shape linking the related entities.



Relationship (Diamond)
Connects two entities using a diamond shape and connecting lines.

Relationship Type	Meaning	Example
One-to-One (1:1)	One record in Entity A relates to exactly one record in Entity B.	One *Person* has one *Passport*.
One-to-Many (1:N)	One record in Entity A relates to many records in Entity B.	One *Department* has many *Employees*.
Many-to-Many (M:N)	Many records in Entity A relate to many records in Entity B.	Many *Students* enroll in many *Courses*.

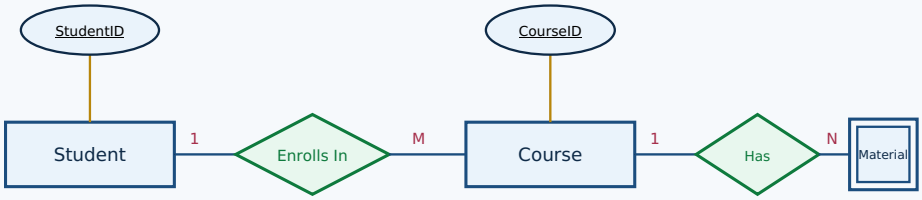
Cardinality — Representing Relationship Counts

Cardinality tells us exactly how many instances of one entity can/must relate to instances of another entity. It's usually written near the connecting line as `1`, `N`, or `M` (or as crow's-foot symbols in some notations).

Reads as: **One** Department **Has Many (N)** Employees — a One-to-Many relationship.

5. Combined Example — Putting It All Together

MINI ER DIAGRAM



Legend: Rectangle = Entity | Double Rectangle = Weak Entity | Oval = Attribute
Underlined = Key | Diamond = Relationship | 1 / M = Cardinality

A **Student** (identified by the key `StudentID`) **Enrolls In** many **Courses** (1:M relationship), and each **Course Has** many **Materials** — a weak entity that depends on Course.

Summary of Day 19: Learned how normalization (1NF → 5NF) removes redundancy and anomalies from tables, and how ER modeling (entities, attributes, relationships, cardinality) is used to visually design a database before building it in MySQL.